

目 录

第 1 章 实验概述	1
1.1 实验目的	1
1.2 实验环境	1
第 2 章 实验原理	2
2.1 向量点积算法	2
2.2 MIPS 汇编语言实现	2
2.3 定向 (Forwarding) 技术	2
2.4 静态调度优化	3
第 3 章 实验内容和步骤	4
3.1 实验内容	4
3.2 向量点积程序实现	4
3.2.1 程序实现	4
3.2.2 程序部分说明	5
3.2.3 数据部分说明	6
3.2.4 程序执行结果	6
3.3 优化程序代码	8
3.3.1 优化思路	8
3.3.2 优化程序实现	9
3.3.3 优化程序执行结果	10
3.4 代码性能分析比较结果	11
第 4 章 实验总结与心得体会	12

第 1 章 实验概述

1.1 实验目的

1. 通过实验熟悉实验 1 和实验 2 的内容。
2. 增强汇编语言编程能力。
3. 学会使用模拟器中的定向功能进行优化。
4. 了解对代码进行优化的方法。

1.2 实验环境

表 1.1 实验环境

配置项	参数
操作系统	Windows 11 24H2
仿真工具	MIPSSim v1.0.0 64-bit

第 2 章 实验原理

2.1 向量点积算法

向量点积 (Dot Product) 是两个等长向量对应元素乘积之和的计算。C 语言的实现方法如下:

```
1 int dot_product_func(int *a, int *b, int n) {  
2     int len = n, ans = 0;  
3     for (int i = 0; i < len; i++)  
4         ans += a[i] * b[i];  
5     return ans;  
6 }
```



2.2 MIPS 汇编语言实现

MIPS 汇编语言是一种简化的 RISC 架构,用于进行低级编程。本实验中,需要编写汇编程序来计算两个向量的点积,其实现的基本思路如下:

- **数据加载:** 使用 LW 指令从内存中加载两个向量的元素到寄存器中。
- **循环控制:** 每次迭代从两个数组中读取一对元素加载到寄存器中,进行乘法运算,并将结果累加到一个结果寄存器中,直到处理完所有元素。

2.3 定向 (Forwarding) 技术

定向技术通过将流水线寄存器中的中间计算结果直接“前递”到需要它的指令的 EX 段输入端,避免不必要的停顿,优化数据的流动,从而提升 CPU 的性能。在没有定向的情况下,某些指令需要等待前一指令完成并将结果写回寄存器后才能继续执行,这会导致停顿。而启用定向后,相关指令可以直接使用前一指令的结果,无需等待写回寄存器,从而减少停顿周期数。

2.4 静态调度优化

静态调度是在编译期（或手动编写时）重新排列指令顺序，以避免数据相关或资源冲突的技术。核心思想是将不相关的指令插入到可能导致停顿的位置，使流水线保持连续工作。典型的优化策略包括：

1. **重排相邻指令**：将与源数据无关的其他指令插入到 load 或运算指令之后，利用原本需要停顿的空隙执行有效工作。
2. **延迟槽填充**：将无害指令填入分支指令后的延迟槽，减少分支带来的性能损失。
3. **循环展开**：通过复制循环体减少循环控制开销，同时增加可用指令数以掩盖延迟。

第3章 实验内容和步骤

3.1 实验内容

1. 自行编写一个计算两个向量点积的汇编程序,该程序要求可以实现求两个向量点积计算后的结果。
 - 向量的点积: 假设有两个 n 维向量 a 、 b , 则 a 与 b 的点积为: $a \cdot b = a_1b_1 + a_2b_2 + \dots + a_nb_n$ 。
 - 两个向量元素使用数组进行数据存储, 要求向量的维度不得小于 10。
2. 启动 MIPSsim。
3. 载入自己编写的程序, 观察流水线输出结果。
4. 使用定向功能再次执行代码, 与刚才执行结果进行比较, 观察执行效率的不同。
5. 采用静态调度方法重排指令序列, 减少相关, 优化程序
6. 对优化后的程序使用定向功能执行, 与刚才执行结果进行比较, 观察执行效率的不同。
 - 注意: 不要使用浮点指令及浮点寄存器。
 - 使用 TEQ \$r0 \$r0 结束程序。

3.2 向量点积程序实现

3.2.1 程序实现

我编写的向量点积实现的汇编实现如下, 注释在代码中已经给出:

```
1  .text ▲Assembly (x86_64)
2  main:
3  ADDIU $r1, $r0, a      # 将向量a的地址加载到寄存器$r1
4  ADDIU $r2, $r0, b      # 将向量b的地址加载到寄存器$r2
5  ADDIU $r3, $r0, n      # 将向量维度n的地址加载到寄存器$r3
```

```

6  BGEZAL $r0, dot_product_func # 跳转到计算点积的函数
7  NOP                               # 占位指令，等待函数返回
8  TEQ $r0, $r0                      # 结束程序
9
10 dot_product_func:
11 LW $r3, 0($r3)                    # 从内存中加载向量的维度n
12 ADD $r4, $r0, $r0                 # 初始化循环索引i为0
13 ADD $r5, $r0, $r0                 # 初始化累加器$r5为0，用于存储结果
14 loop:
15 LW $r6, 0($r1)                    # 从向量a中加载第i个元素到$r6
16 LW $r7, 0($r2)                    # 从向量b中加载第i个元素到$r7
17 MUL $r8, $r6, $r7                # 计算a[i] * b[i]，结果存储到$r8
18 ADD $r5, $r5, $r8                 # 将计算结果累加到累加器$r5中
19 ADDIU $r1, $r1, 4                 # 将向量a的地址指针移动到下一个元素
20 ADDIU $r2, $r2, 4                 # 将向量b的地址指针移动到下一个元素
21 ADDIU $r4, $r4, 1                 # 循环索引i加1
22 BNE $r4, $r3, loop               # 如果i < n，继续循环
23 JR $r31                          # 返回主程序
24
25 # 数据部分
26 .data
27 a: .word 2, 4, 6, 8, 10, 12, 14, 16, 18, 20 # 向量a的数据
28 b: .word 1, 3, 5, 7, 9, 11, 13, 15, 17, 19 # 向量b的数据
29 n: .word 10                        # 向量的维度n

```

3.2.2 程序部分说明

程序主要分为两部分：主程序 main 和计算点积的函数 dot_product_func。主程序负责初始化寄存器并调用函数，而函数部分则实现了点积的计算逻辑，包括数据加载、循环控制和结果累加。

1. **初始化**：主程序首先将向量 a 和 b 的地址以及向量维度 n 的地址加载到寄存器 \$r1、\$r2 和 \$r3 中，然后通过 BGEZAL 指令跳转到计算点积的函数 dot_product_func。
2. **函数实现**：在 dot_product_func 中，首先从内存中加载向量的维度 n，然后初始化循环索引 \$r4 和累加器 \$r5。

3. **进入循环**: 进入循环后, 每次迭代从向量 a 和 b 中加载对应元素进行乘法运算, 并将结果累加到 \$r5 中。
4. **循环索引更新**: 循环索引 \$r4 每次迭代加 1。
5. **循环结束判断**: 循环通过 BNE 指令判断是否继续, 若 \$r4 不等于 \$r3 (即 $i < n$), 则继续循环。否则通过 JR \$r31 返回主程序。
6. **程序结束**: 主程序在函数返回后执行 TEQ \$r0, \$r0 指令结束程序。

这里我通过 BGEZAL + JR \$r31 的方式实现了**函数调用和返回**, 其中:

- BGEZAL \$r0, dot_product_func: 这是一个条件跳转指令, 表示如果 \$r0 大于或等于零 (在这里 \$r0 始终为零), 则跳转到 dot_product_func 标签处执行函数, 并将返回地址存储在链接寄存器 \$r31 中。
- JR \$r31: 这是一个无条件跳转指令, 表示跳转到 \$r31 寄存器中存储的地址处执行, 这里用于函数返回。

3.2.3 数据部分说明

数据部分定义了两个向量 a 和 b, 以及向量的维度 n。每个向量包含 10 个元素, 维度 n 的值为 10, 表示每个向量有 10 个元素。

3.2.4 程序执行结果

3.2.4.1 未开启定向的执行结果

载入并执行程序后, 执行结果统计如图 3.1 所示, 部分时间周期图如图 3.2 所示, 程序共执行了 163 个周期, 其中发生了 60 次 RAW 停顿。整体停顿周期占周期总数的 44.17%, 主要由**数据相关**引起。

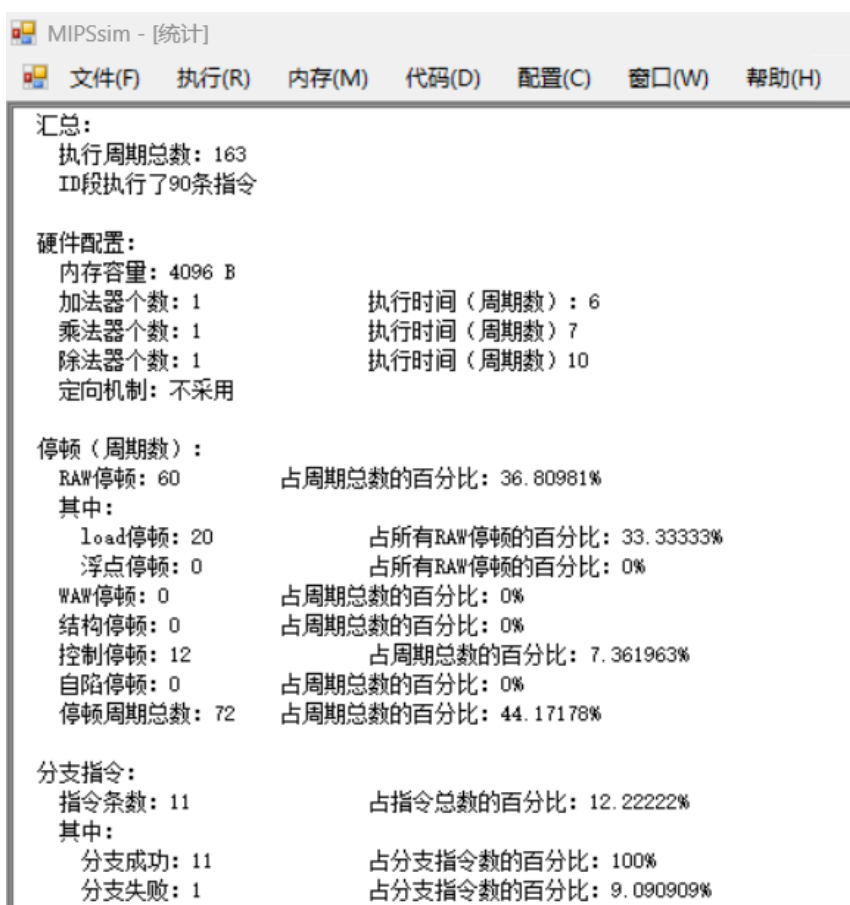


图 3.1 未开启定向的执行结果统计

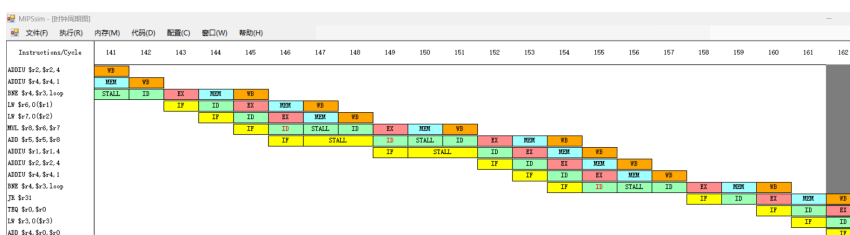


图 3.2 未开启定向的部分时间周期图

3.2.4.2 开启定向的执行结果

开启定向功能后再次执行程序，执行结果统计如图 3.3 所示，部分时间周期图如图 3.4 所示，程序共执行了 123 个周期，其中发生了 20 次 RAW 停顿。整体停顿周期占周期总数的 26.02%，相比未开启定向时大幅减少，说明定向技术有效地优化了数据流动，提升了程序的执行效率。

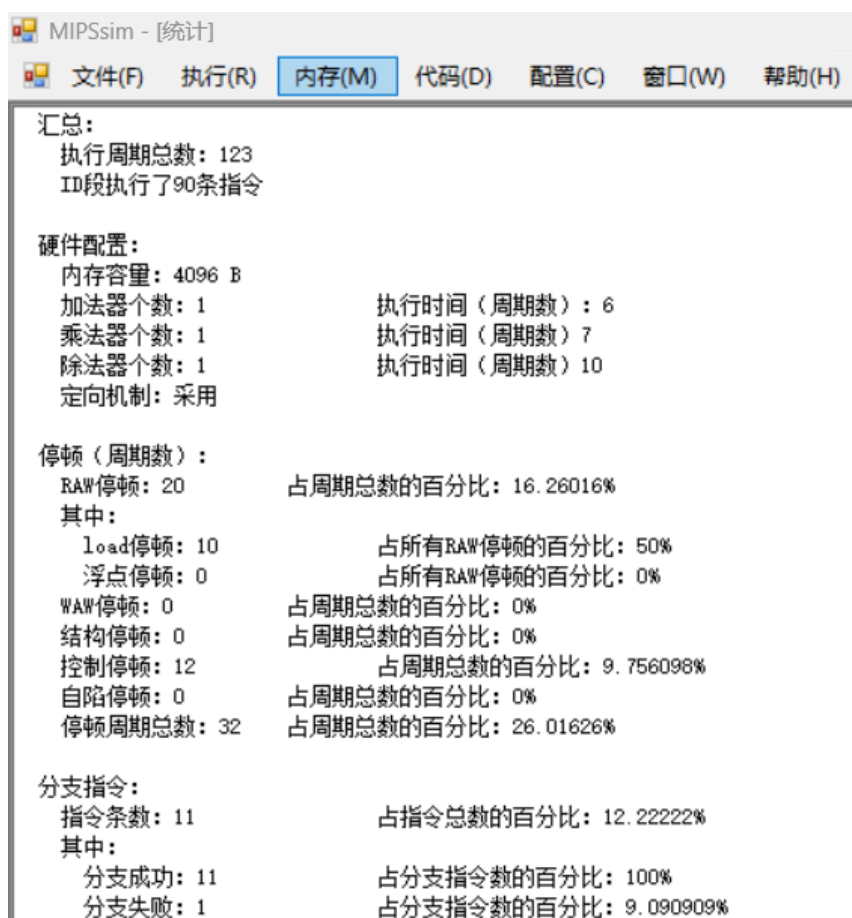


图 3.3 开启定向的执行结果统计

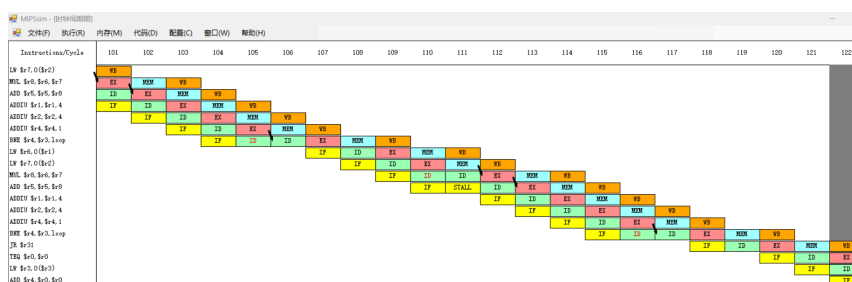


图 3.4 开启定向的部分时间周期图

3.3 优化程序代码

3.3.1 优化思路

针对上述程序中的数据相关问题，我采用了**静态调度**的方法对指令序列进行了重排，以减少 RAW 停顿的发生。原先的程序中：

- 每次迭代中，乘法指令 MUL 依赖于前一条指令加载的操作数；
- 累加指令 ADD 依赖于前一条乘法指令的结果。

因此，通过**合理安排操作数加载、乘法指令、累加指令以及指针和索引更新的顺序**，能减少指令之间的直接数据依赖，从而降低停顿周期数。

3.3.2 优化程序实现

1	<code>.text</code>	Assembly (x86_64)
2	<code>main:</code>	
3	<code>ADDIU \$r1, \$r0, a</code>	# 将向量a的地址加载到寄存器\$r1
4	<code>ADDIU \$r2, \$r0, b</code>	# 将向量b的地址加载到寄存器\$r2
5	<code>ADDIU \$r3, \$r0, n</code>	# 将向量维度n的地址加载到寄存器\$r3
6	<code>BGEZAL \$r0, dot_product_func</code>	# 跳转到计算点积的函数
7	<code>NOP</code>	# 占位指令，等待函数返回
8	<code>TEQ \$r0, \$r0</code>	# 结束程序
9		
10	<code>dot_product_func:</code>	
11	<code>LW \$r3, 0(\$r3)</code>	# 从内存中加载向量的维度n
12	<code>ADD \$r4, \$r0, \$r0</code>	# 初始化循环索引i为0
13	<code>ADD \$r5, \$r0, \$r0</code>	# 初始化累加器\$r5为0，用于存储结果
14	<code>loop:</code>	
15	# 核心修改部分：对指令顺序进行重排，减少数据相关	
16	<code>LW \$r6, 0(\$r1)</code>	# 从向量a中加载第i个元素到\$r6
17	<code>LW \$r7, 0(\$r2)</code>	# 从向量b中加载第i个元素到\$r7
18	<code>ADDIU \$r1, \$r1, 4</code>	# 将向量a的地址指针移动到下一个元素
19	<code>ADDIU \$r2, \$r2, 4</code>	# 将向量b的地址指针移动到下一个元素
20	<code>MUL \$r8, \$r6, \$r7</code>	# 计算a[i] * b[i]，结果存储到\$r8
21	<code>ADDIU \$r4, \$r4, 1</code>	# 循环索引i加1
22	<code>ADD \$r5, \$r5, \$r8</code>	# 将计算结果累加到累加器\$r5中
23	# 核心修改部分结束	
24	<code>BNE \$r4, \$r3, loop</code>	# 如果i < n，继续循环
25	<code>JR \$r31</code>	# 返回主程序
26		

```

27 # 数据部分
28 .data
29 a: .word 2, 4, 6, 8, 10, 12, 14, 16, 18, 20 # 向量a的数据
30 b: .word 1, 3, 5, 7, 9, 11, 13, 15, 17, 19 # 向量b的数据
31 n: .word 10 # 向量的维度n

```

3.3.3 优化程序执行结果

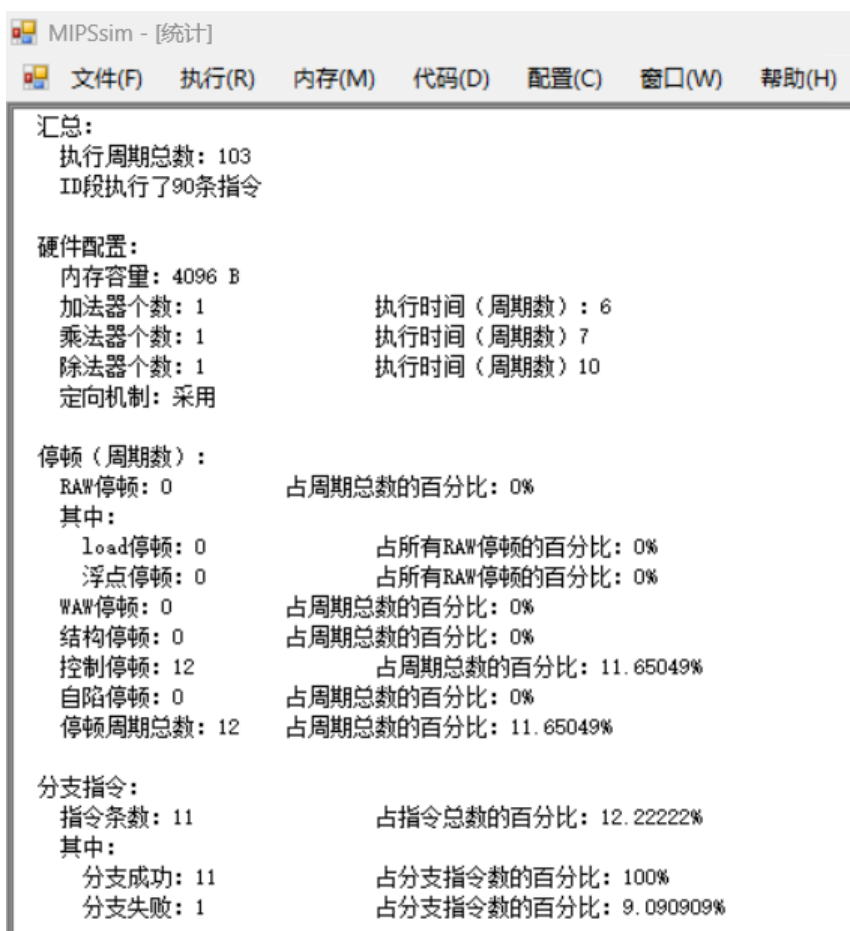


图 3.5 优化程序执行结果统计

开启定向功能后执行优化后的程序，执行结果统计如图 3.5 所示，部分时间周期图如图 3.6 所示，程序共执行了 103 个周期，其中未发生 RAW 停顿，仅有控制相关造成共 12 次停顿，占周期总数的 11.65%。

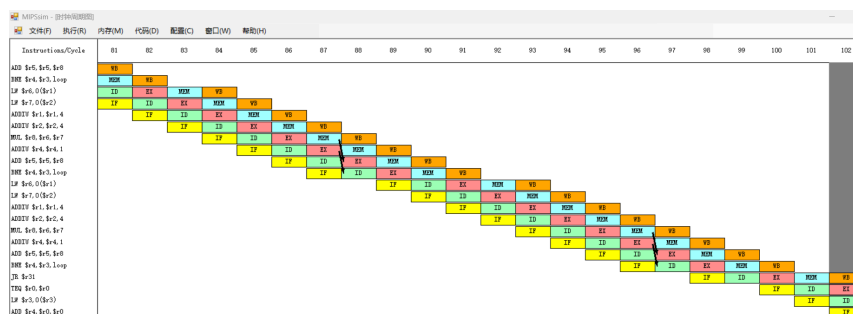


图 3.6 优化程序的部分时间周期图

3.4 代码性能分析比较结果

优化后的程序相比优化前的版本，执行周期数从 163 个减少到 103 个，停顿占比从 44.17% 降至 11.65%，效率提升至 $163/103 \approx 1.58$ 倍；

而相比仅开启定向的版本，执行周期数从 123 个减少到 103 个，停顿占比从 26.02% 降至 11.65%，效率提升至 $123/103 \approx 1.19$ 倍。

优化后的程序有了显著提升，主要得益于以下两点：

- 通过定向技术优化了数据流动；
- 通过静态调度优化指令顺序有效地减少了数据相关引起的停顿；

第4章 实验总结与心得体会

在本次实验中，我完成了向量点积功能的汇编代码编写，并进行了两方面的优化：（1）定向技术——开启后 RAW 停顿从 60 次降至 20 次；（2）静态调度——通过重排指令顺序，将 RAW 停顿进一步降至 0 次，总周期从 163 减少到 103。

通过本次实验，我进一步掌握了 MIPSsim 模拟器的使用方法，同时对模拟器所支持的 MIPS 指令集和作用有了更输入的了解，对执行中 MIPS 程序行为有了更好的理解，并通过指令实现了循环和函数调用过程，取得了较大的收获。